What are researchers doing right now to make large language model inference faster and cheaper?

GraphIV — deep research

August 27, 2026

Abstract

(judge output unparseable)

1 Scaling Efficiency: A Survey of Modern Large Language Model Inference Optimization

The unprecedented growth in the parameter counts of large language models (LLMs) has introduced a critical bottleneck in the deployment of artificial intelligence: the prohibitive cost and latency of inference. As these models become central to global digital infrastructure, the demand for high-throughput, low-latency responses clashes with the massive memory and computational requirements of transformer architectures. Solving the efficiency problem is essential not only for reducing the environmental and economic footprint of AI but also for enabling real-time interaction and democratizing access to advanced reasoning capabilities across diverse hardware environments.

This review traces the evolution of inference optimization from fundamental model-level reductions to sophisticated, system-wide architectures. We begin by examining foundational techniques in model compression and sparsity, which aim to reduce the inherent size and density of neural networks. We then explore algorithmic acceleration through methods like speculative decoding and early-exiting, followed by systemic strategies for optimizing throughput via advanced batching, scheduling, and memory management. The scope expands to include the structural shifts enabled by distributed and disaggregated inference architectures and the critical role of hardware-software co-design and specialized accelerators. We further address the constraints of edge and mobile deployment, before finally looking toward the frontier of the field: the development of reasoning-aware and adaptive scaling laws that allow models to dynamically allocate compute based on task complexity.

1.1 Foundational Techniques: Model Compression and Sparsity

Current research into accelerating large language model (LLM) inference focuses on two primary axes: reducing the computational footprint of the model architecture itself and minimizing the volume of input data processed during the prefill and decoding phases. As summarized in [1], these efforts encompass a broad spectrum of techniques including pruning, quantization, knowledge distillation, and compact architecture design to optimize network structure and attention computation.

A significant portion of recent work targets the model's internal parameters through pruning. Structural pruning approaches, such as LLM-Pruner, utilize a three-stage process (Discovery, Estimation,

and Recovery) to remove interdependent structures, achieving a 20% reduction in parameters ([2]). This contrasts with weight-centric methods like Wanda, which focuses on pruning network weights by considering both weights and activations ([3]). While structural pruning aims for architectural efficiency, sparsity-based methods like Deja Vu introduce contextual sparsity by targeting the MLP and Attention blocks to reduce token generation latency ([4]).

Another distinct research direction addresses the inefficiency of long input contexts. Rather than modifying the model weights, these algorithmic techniques compress the prompt to save tokens and context window usage. Selective Context optimizes context efficiency by filtering redundant content using self-information ([5]), whereas LLMLingua focuses on compressing the instruction, demonstrations, and questions to reduce inference costs via higher compression ratios ([6]).

| Method | Target Component | Optimization Category | Primary Resource Saved | | :— | :— | :— | :— | | [6] | Input prompts | Compression | Tokens (sequence length) | | [5] | Input context | Compression | Context window usage | | [4] | MLP/Attention blocks | Sparsity | Latency | | [2] | Interdependent structures | Compression | Parameters (20% removal) | | [7] | KV-cache/Phase execution | System | Throughput, cost, and power |

While compression and sparsity techniques primarily operate at the algorithmic level to reduce the mathematical complexity of the model, system-level innovations like Splitwise shift the focus toward execution efficiency. Splitwise employs phase splitting—separating prompt computation from token generation onto distinct machines—to optimize throughput, cost, and power ([7]).

The literature reveals a tension between these approaches: prompt compression ([6], [5]) provides a "shallow" efficiency gain by reducing input volume without altering the model, whereas structural pruning ([2]) and sparsity ([4]) offer "deep" efficiency by fundamentally altering how the model processes information. However, while prompt compression is highly effective for long-context scenarios, it does not reduce the inherent computational cost of the model's weights, a gap that system-level architectural changes ([7]) and weight-based pruning aim to bridge.

Beyond standard pruning, architectural innovations such as skipping specific attention sublayers can yield significant speedups in single-token generation ([8]). Furthermore, algorithmic approaches like the Heavy-Hitter Oracle (H2O) optimize the KV cache to reduce the memory footprint ([9]), while evolution-based search methods can be used to discover more efficient model architectures ([10]).

1.2 Algorithmic Acceleration: Speculative Decoding and Early-Exiting

Current research into accelerating large language model (LLM) inference focuses heavily on reducing the overhead of the autoregressive decoding process through two primary paradigms: speculative decoding and early-exiting mechanisms. Speculative decoding seeks to improve throughput by using a smaller, faster "drafter" model to predict multiple tokens, which are then verified in parallel by the target LLM [[11]].

The evolution of speculative decoding shows a shift from basic draft-verify paradigms toward more complex architectural integrations. While the foundational approach focuses on the autoregressive decoding process to reduce walltime [[11]], subsequent works have refined the drafter's role. For instance, the "Big Little Decoder" approach achieves speedups ranging from 1.85x to 2.12x by optimizing the generation process [[12]]. More recent frontier research extends this to multilingual contexts, utilizing specialized drafters to save memory bandwidth during the draft-verify-accept cycle [[13]].

A significant trend in the literature is the transition from simple linear token prediction to structural optimizations. SpecInfer moves beyond single-sequence drafting by implementing tree-based speculative inference and verification, which targets end-to-end latency and reduces overall computational requirements at a system level [[14]]. Similarly, the Lookahead framework introduces a multi-branch strategy paired with a Verification and Accept (VA) process to accelerate decoding while maintaining lossless generation accuracy [[15]]. This highlights a critical tension in the field: the need to increase speed without sacrificing the mathematical integrity of the model's output.

While speculative decoding optimizes the *sequence* of generation, early-exiting algorithms attempt to optimize the *depth* of computation. Unlike the draft-verify paradigm, which adds a secondary model, early-exiting employs architectural and algorithmic co-design to reduce the computational cost per token [[16]]. This is achieved through lightweight routers, deep parameter sharing, and parallel decoding mechanisms that allow the model to exit the computation early if a certain confidence threshold is met [[16]].

| Method | Optimization Category | Implementation Level | Primary Target | | :— | :— | :— | :— | | [11] | Algorithmic | Algorithmic | Autoregressive decoding | | [12] | Arch./Algorithmic | Algorithmic | Autoregressive generation | | [14] | Architectural | System | Token decoding/generation | | [15] | Architectural | Algorithmic | LLM inference decoding | | [13] | Algorithmic | Algorithmic | Token generation | | [16] | Arch./Algorithmic | Alg./Arch. | Model architecture/decoding |

The literature reveals a clear divergence in implementation strategy. Speculative decoding methods, such as those in [[11]], [[12]], [[14]], [[15]], and [[13]], primarily focus on the interaction between a drafter and a verifier to bypass the sequential nature of autoregression. In contrast, early-exiting [[16]] fundamentally alters the model's internal execution path, trading off model depth for throughput. While speculative decoding is highly effective for reducing latency through parallel verification, it requires the management of additional model parameters (the drafter), whereas early-exiting aims to reduce the total parameters and computation required per inference step.

Speculative decoding is being refined through dynamic tree construction techniques that can achieve up to 5.2x speedups over vanilla autoregressive methods ([17]). Additionally, early-exiting strategies, such as LITE, leverage instruction tuning to enable decoding from intermediate layers ([18]).

1.3 Optimizing Throughput: Batching, Scheduling, and Memory Management

Current research into maximizing LLM throughput focuses on three interconnected pillars: optimizing batching strategies, refining request scheduling, and managing the memory overhead of the KV cache. While early approaches prioritized compression to fit models into limited hardware, recent frontier work shifts toward dynamic, runtime-aware orchestration of computational resources.

A primary bottleneck in inference is the inefficient utilization of GPU resources due to "computational bubbles" and rigid batching. BATON addresses this by implementing dynamic re-batching through vector shaping and P&D decoupling, which improves query processing throughput by 1.29–1.75x [[19]]. This contrasts with more traditional scheduling methods, such as the multi-bin batching algorithm [[20]] or the simulated annealing-based SLO-aware scheduler, which prioritizes Service Level Objective (SLO) attainment and latency reduction [[21]]. While BATON targets the structural efficiency of the batching process itself, SLO-aware scheduling focuses on the dispatching logic to reduce average latency by up to 31.6% and improve SLO attainment by up to 5% [[21]].

Memory management, specifically the KV cache, serves as a critical dimension for both throughput and latency. Researchers have moved from static allocation to highly dynamic, predictive, or reuse-

oriented strategies. S^3 utilizes a fine-tuned DistalBERT model to predict sequence lengths, allowing for more efficient KV cache memory allocation [[22]]. In contrast, SGLang optimizes throughput by enabling KV cache reuse across multiple generation calls at runtime, which can accelerate common LLM tasks by up to 5x [[23]]. More recent algorithmic approaches, such as the memory-aware and SLA-constrained dynamic batching proposed in [24], attempt to unify these concerns by simultaneously optimizing batch size configuration and KV cache capacity to maximize tokens processed per second [[24]].

| Method | Primary Optimization Target | Efficiency Metric | | :— | :— | :— | | [23] | KV cache reuse | Up to 5x vs. common LLM tasks | | [19] | Batch-wise process | 1.29–1.75x vs. baseline throughput | | [21] | Request scheduling | Up to 31.6% vs. baseline latency |

The literature reveals a transition from coarse-grained optimizations to fine-grained, system-algorithm co-designs. Early work like FlexGen focused on compression via group-wise quantization and sparse attention to reduce I/O costs [[25]], whereas contemporary methods like BATON and the memory-aware batching in [24] treat batching and memory as a joint optimization problem [[19], [24]]. However, a tension remains between maximizing raw throughput and maintaining strict latency guarantees; while BATON and S^3 prioritize the former [[19], [22]], SLO-aware schedulers suggest that throughput gains are secondary to meeting user-defined service constraints [[21]].

System-level optimizations are increasingly focusing on task scheduling, such as the ScheInfer strategy which utilizes weight tensor slicing to improve latency on single GPU systems ([26]). Comprehensive surveys highlight that modern inference engines must balance parallelism, compression, and caching to maximize throughput ([27], [28]).

1.4 Distributed and Disaggregated Inference Architectures

Recent research has shifted from monolithic inference toward distributed and disaggregated architectures to mitigate resource contention between different inference phases. A primary focus is the separation of the prefill and decode phases, a strategy designed to optimize Time to First Token (TTFT), Job Completion Time (JCT), and performance-per-dollar ([29]). This disaggregation has evolved from simple phase separation to more complex, multi-stage pipelines. For instance, HydraInfer extends this to a three-stage Encode-Prefill-Decode (EPD) disaggregation specifically to support multimodal LLM serving ([30]).

While disaggregation improves phase-specific efficiency, it introduces significant inter-node communication overhead. KVDirect addresses this bottleneck by optimizing the KV cache transfer process through a "Connect, Transfer, Complete" mechanism to reduce round-trip communication and synchronization ([31]). Similarly, in multi-vendor GPU environments, researchers have moved toward joint optimization of parallel strategies and instance ratio allocation to maximize resource utilization and inference speed ([32]).

| Method | Target Component | Optimization Category | Primary Metric | | :— | :— | :— | :— | | [29] | Prefill and decode phases | Architectural/System-level scheduling | TTFT, JCT, and perf/$ | | [31] | KV cache transfer | Architectural (disaggregated inference) | Communication overhead | | [30] | Encode, Prefill, and Decode stages | Architectural (Disaggregated scheduling) | System throughput and service capacity | | [32] | Prefill and Decoding stages | Architectural (disaggregated inference) | Resource utilization and speed |

While the aforementioned methods optimize the internal inference pipeline, other distributed approaches focus on model placement and workload distribution. SplitLLM utilizes collaborative

execution between servers and clients to optimize throughput and reduce server-side workload, specifically targeting FLOP or GPU memory savings ([33]). This contrasts with purely distributed serving systems that utilize skip-join MLFQ schedulers and proactive KV cache management to optimize JCT and GPU memory capacity ([34]).

The complexity of managing these distributed components across heterogeneous environments is addressed by GUIDE, which employs simulation-based optimization to navigate the multi-dimensional parameter space of hardware platforms, frameworks, and parallel strategies ([35]). At the micro-architectural level, optimizations like FlashDecoding++ complement these high-level architectural shifts by targeting the attention computation itself, using asynchronized softmax with a unified max value and flat GEMM optimization to improve raw inference speed ([36]).

Recent literature provides a holistic view of the evolving landscape of LLM inference serving, emphasizing the need to manage the massive memory overhead inherent in generative models ([37]).

1.5 Hardware-Software Co-Design and Specialized Accelerators

Current research into accelerating Large Language Model (LLM) inference is shifting from purely algorithmic optimizations toward deep hardware-software co-design to overcome the "memory wall" and data movement bottlenecks. While algorithmic approaches like Multi-query attention (MQA), Grouped-query attention (GQA), and Mixture of Experts (MoE) focus on reducing the computational and memory footprint of attention mechanisms and network execution [[38]], recent frontier work suggests that hardware-level interventions are necessary to realize these theoretical gains.

A significant trend involves integrating computation closer to memory to mitigate data movement energy. HALO utilizes 2.5D heterogeneous integration, combining HBM-based Compute-in-DRAM (CiD) with on-chip analog Compute-in-Memory (CiM) to target low-batch inference [[39]]. This memory-centric approach achieves substantial performance leaps, demonstrating up to a 2.5x speedup over CENT and 18x over AttAcc [[39]]. Similarly, SAIL addresses memory bandwidth constraints by employing near-memory computing using SRAM as a ping-pong buffer and utilizing lookup-table-based GEMV [[40]]. SAIL's system-level approach results in a 10.7x speedup and 19.9x improvement in cost efficiency compared to its baseline [[40]].

Other researchers focus on the synergy between compression and architectural efficiency. AccLLM employs an algorithm-hardware co-design strategy that combines semi-structured pruning, k-shaped attention, and W2A8KV4 quantization to reduce computational complexity and memory access overheads [[41]]. This contrasts with purely architectural efforts like Inferflow, which focuses on a modular, highly configurable inference engine framework to optimize latency through atomic build-blocks [[42]], or studies focusing on scaling strategies through expert and tensor parallelism [[43]].

| Method | Primary Optimization Target | Implementation Level | Key Efficiency Metric | | :— | :— | :— | :— | | [39] | HBM-based CiD and analog CiM | Hardware | 2.5x speedup vs CENT; 18x vs AttAcc | | [40] | SRAM near-memory computing | System/Hardware | 10.7x speedup; 19.9x cost efficiency vs baseline | | [41] | Pruning and W2A8KV4 quantization | Algorithm-Hardware | Throughput (via reduced complexity) |

The literature reveals a tension between general-purpose architectural modularity and highly specialized, domain-specific hardware. While modular engines like Inferflow [[42]] offer flexibility, they may lack the extreme efficiency gains seen in specialized accelerators like HALO [[39]] or SAIL [[40]], which achieve orders-of-magnitude improvements by fundamentally altering how data moves between memory and compute units. However, the extreme specialization of near-memory or CiM-

based approaches may introduce complexity in deployment compared to the algorithmic-heavy optimizations (e.g., GQA/MoE) that can be implemented on existing AI accelerators [[38]].

Hardware-centric perspectives are becoming essential, particularly in understanding the unique acceleration requirements of generative LLMs compared to traditional encoder models ([44]).

1.6 Edge and Mobile Deployment: Resource-Constrained Inference

Current research into edge and mobile LLM deployment focuses on three primary axes: algorithmic acceleration via speculative execution, neuron-level activation management, and system-level hardware/memory optimizations.

Algorithmic speedups are frequently pursued through speculative decoding to mitigate the memory wall. LLMCad [2309.04255] utilizes a speculative generation pipeline, employing a memory-resident model as a generator and a target model as a verifier to improve latency. Similarly, research into specialized drafters [2406.16758] refines this draft-verify-accept paradigm to optimize inference time and memory bandwidth, particularly for multilingual tasks. While both approaches target the generation process at an algorithmic level, they represent a shared shift toward decoupling the drafting and verification stages to bypass bandwidth bottlenecks.

A second approach targets the inherent sparsity of model activations. PowerInfer [2312.12456] exploits the locality of "hot" versus "cold" neurons to optimize GPU memory usage on consumer-grade hardware. This concept is significantly advanced in PowerInfer-2 [2406.06282], which transitions from GPU-centric to smartphone-centric deployment. By implementing a polymorphic neuron engine that decomposes matrix computations into fine-grained neuron clusters, PowerInfer-2 achieves 68 tokens/s on a smartphone and reduces memory requirements by nearly 40% for 7B models. This represents a shift from purely algorithmic neuron management to a system-level implementation capable of handling the extreme constraints of mobile hardware.

System-level optimizations focus on the specific hardware-software interface of mobile devices. Transformer-Lite [2403.20041] optimizes mobile GPU performance through a multi-faceted approach, including FP4 quantization (M0E4), sub-tensor-based KV cache management, and symbolic expression-based dynamic shape support. In contrast, MNN-LLM [2506.10443] addresses the memory bottleneck by combining quantization with a DRAM-Flash hybrid storage strategy, specifically targeting high-computation operators and data layout to improve both speed and memory efficiency. While Transformer-Lite focuses on the efficiency of the GPU execution and cache management, MNN-LLM prioritizes the management of the memory hierarchy itself.

Finally, researchers are exploring collaborative paradigms that move beyond single-device constraints. PICE [2501.09367] introduces a progressive inference paradigm for cloud-edge networks. Unlike the previous methods that optimize a single device's resources, PICE utilizes a two-stage modeling approach—cloud-based sketch generation followed by edge-based parallel extension—to achieve a 1.5x to 2x enhancement in throughput and up to a 43% reduction in latency. This collaborative approach suggests that the future of edge deployment may rely as much on network orchestration as on local device optimization.

None

1.7 The Frontier: Reasoning-Aware and Adaptive Scaling Laws

Recent research has shifted from general scaling towards optimizing the specific relationship between model architecture, reasoning depth, and inference efficiency. This frontier can be categorized into three distinct approaches: architectural co-optimization, reasoning-aware training, and dynamic model selection.

One prominent direction involves refining scaling laws to prioritize inference efficiency. While traditional scaling focuses on parameter count and token volume, [45] proposes modified Chinchilla scaling laws that co-optimize these parameters alongside model architecture to minimize inference latency. Similarly, [46] moves beyond parameter scaling to perform architecture searches, specifically varying hidden sizes and the ratio of MLP-to-attention layers to maximize inference throughput in decoder-only transformers. These methods suggest that the "optimal" model is no longer just a function of data and parameters, but is deeply tied to the structural efficiency of the architecture itself.

A second approach focuses on the computational cost of reasoning, particularly the length of Chain-of-Thought (CoT) sequences. Rather than altering the architecture, [47] optimizes the training objective by using a monotonic function to penalize response length, thereby improving token-efficiency during inference. This contrasts with distillation-based methods like [48], which seeks to compress reasoning capabilities by distilling knowledge from Transformers into more efficient architectures, such as Mamba, achieving a 2.5x speedup in inference time. Furthermore, [49] introduces "Gradual Compaction" (PTL) to reduce FLOPs by progressively compacting parameters, offering a different path to efficiency through structural reduction rather than objective-based length control.

Finally, researchers are exploring adaptive inference through model cascading and selection.

| Method | Optimization Category | Primary Efficiency Metric | Key Result/Mechanism | | :— | :— | :— | :— | | [50] | Architectural (Cascading) | Token costs | 60% reduction in GPT-4o usage vs. baseline; 36.46% cost reduction for 8B $\to$ 70B LLaMA | | [47] | Algorithmic (Training) | Token-efficiency | Length-penalized objective function | | [48] | Compression/Distillation | Inference time | 2.5x speedup via Transformer-to-Mamba distillation |

[50] introduces a sophisticated confidence-driven selection mechanism that uses both token probability $P(T)$ and a trained MLP classifier $P(IK)$ to route queries through a hierarchy of LLMs. While this achieves significant reductions in both token usage and computational cost, it introduces new complexities. The reliance of the $P(IK)$ classifier on supervised training for specific datasets limits its generalizability, and the multi-stage escalation process risks increasing end-to-end latency and causing error propagation across the model cascade. This highlights a fundamental tension in the field: while architectural and training-based optimizations ([45], [47]) aim to make individual models leaner, adaptive routing ([50]) attempts to manage the cost of intelligence by intelligently selecting the scale of the model required for a given task.

None

1.8 Further Reading

- [51] (2024) — "A Survey on Efficient Inference for Large Language Models" [survey, cited-by 0 in-topic]
- [52] (2023) — "FrugalGPT: How to Use Large Language Models While Reducing Cost and

Improving Performance" [core, cited-by 0 in-topic]

1.9 Tensions and Open Problems

A primary tension in the current literature concerns the optimal allocation of computational budget: whether intelligence should be scaled through increased inference-time computation or through more efficient model architectures. On one side, researchers argue that intelligence can be augmented by scaling the "thinking" process, using distilled reasoners to expand inference compute for complex tasks ([48]). This view suggests that reasoning is a temporal process that can be stretched to improve accuracy. Conversely, a growing body of work suggests that the focus should remain on the structural efficiency of the model itself, arguing that scaling laws must be reconciled with model architecture to create inherently inference-efficient LLMs ([46]). This debate poses a fundamental question: is reasoning an emergent property of extended compute cycles, or is it a structural property that must be baked into the model's design to remain cost-effective?

A second debate centers on the trade-off between dynamic inference efficiency and the preservation of high-level reasoning capabilities. Proponents of dynamic execution suggest that we can significantly reduce costs by employing early-exiting algorithms ([16]) or by utilizing inference-cost-aware dynamic tree construction ([17]) to bypass unnecessary computations. However, there is growing concern regarding the "reasoning debt" incurred by such methods. While some approaches attempt to mitigate this through gradual compaction strategies to prevent the loss of cognitive depth—a concept described as avoiding the "boiling frog" effect ([49])—it remains unclear if aggressive pruning or early exiting fundamentally compromises the model's ability to handle multi-step logical deductions compared to full-width inference.

Finally, there is a structural tension between centralized serving optimization and the decentralization of intelligence. Much of the recent survey literature focuses on optimizing large-scale inference engines and serving systems to maximize throughput and minimize latency in cloud environments ([37]; [28]). In contrast, a burgeoning subfield argues that the future of LLM deployment lies at the network edge, necessitating entirely different principles for distributed inference that prioritize local resource constraints over global throughput ([53]). This creates a divide between researchers optimizing for "massive-scale efficiency" in the data center and those optimizing for "ubiquitous efficiency" on edge devices.

1.10 Outlook

The frontier of LLM inference is shifting from simple parameter reduction toward "intelligence-aware" efficiency. We are moving away from static model deployment and toward highly adaptive systems that can dynamically modulate their computational depth based on task complexity and available hardware. The next generation of research will likely converge on the intersection of reasoning-centric compression and hardware-aware edge orchestration, seeking to create models that do not just "run faster," but "think smarter" with a strictly bounded energy and latency budget.

Training Language Models to Reason Efficiently — [47] p.2

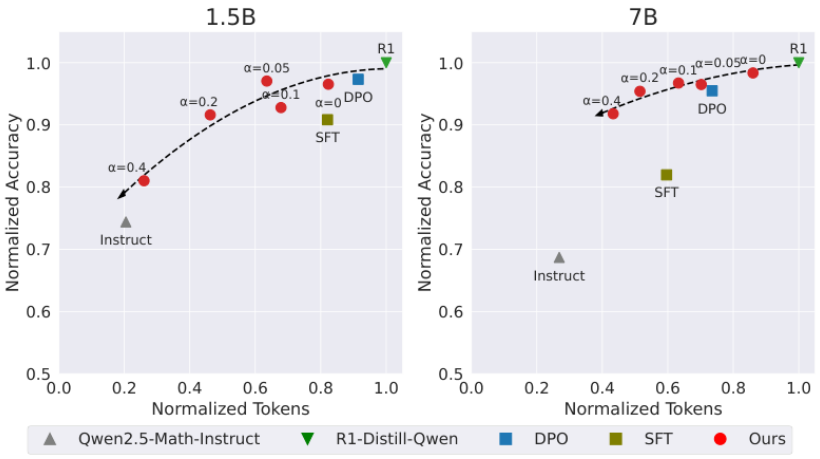


Figure 1: This figure describes the results of our training procedure aggregated on 5 evaluation datasets on both 1.5B and 7B scales. The x-axis represents response length normalized by the performance of the R1-Distilled model and the y-axis represents accuracy normalized by the accuracy of the R1-Distilled model. Each value of α has been aggregated over 3 seeds. Our training reward design allows us to easily and effectively navigate the token-accuracy tradeoff curve. Increasing the value of α generally results in models that have lower response lengths.

The figure consists of two scatter plots (labeled 1.5B and 7B) showing the relationship between Normalized Tokens on the x-axis and Normalized Accuracy on the y-axis. Data points represent different training methods—including Instruct, SFT, DPO, R1-Distill-Qwen, and various α values for the authors' method (red dots)—connected by a dashed black line. The plots convey a token-accuracy tradeoff curve, demonstrating how adjusting the α parameter allows for navigating between response length and accuracy.

Search for Efficient Large Language Models — [10] p.2

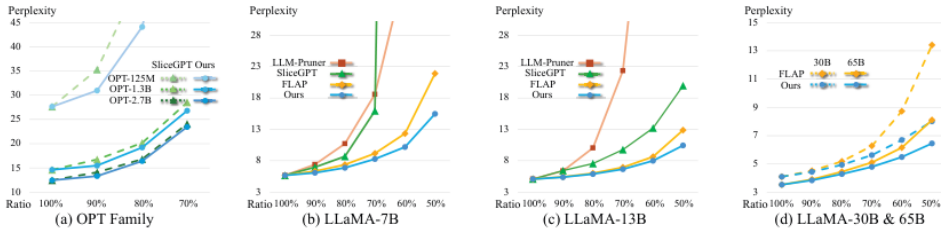


Figure 1: Experiment results of perplexity ↓ on WikiText2 dataset with 2048 sequence length.

The figure consists of four line graphs labeled (a) OPT Family, (b) LLaMA-7B, (c) LLaMA-13B, and (d) LLaMA-30B & 65B, which plot Perplexity on the y-axis against Ratio on the x-axis. Each plot uses colored lines and markers to compare the perplexity performance of different pruning methods (such as SliceGPT, FLAP, and "Ours") across various model scales. The graphs convey that the "Ours" method generally maintains lower perplexity at higher compression ratios compared to the other baseline methods.

LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models — [6] p.3

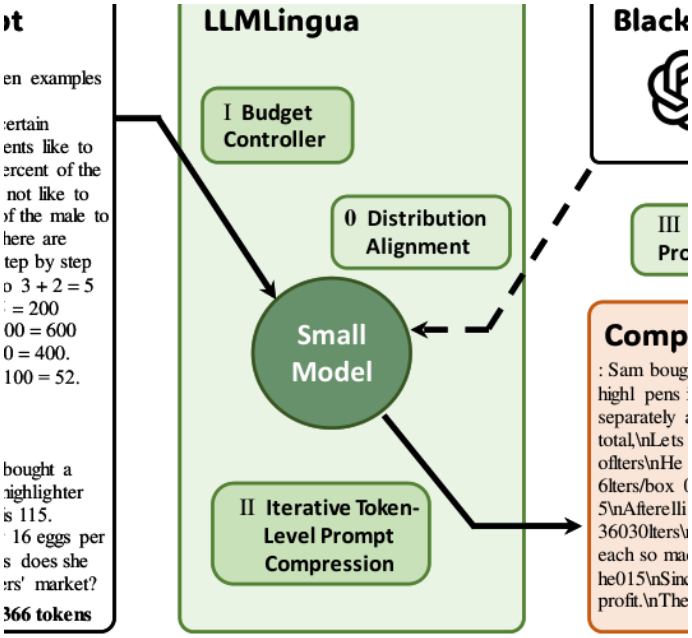


Figure 1: Framework of the proposed approach *LLMLingua*.

The figure presents a flowchart of the LLMLingua framework, illustrating the process of compressing a large prompt into a smaller one for efficient LLM inference. It depicts a workflow where an input prompt passes through a "Budget Controller" and "Distribution Alignment" module to interact with a "Small Model," which then performs "Iterative Token-Level Prompt Compression" to produce a compressed output for a "Black-box LLM."

Confidence-Driven Multi-Scale Model Selection for Cost-Efficient Inference — [50] p.2

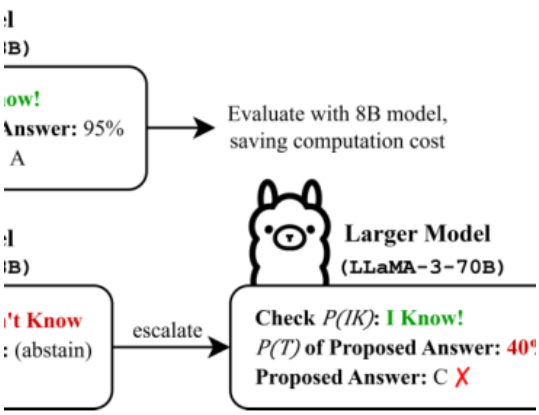


Figure 1: An illustration of our strategy.

The figure uses boxes, arrows, and text labels to illustrate a decision-making strategy for model selection. It depicts a workflow where a model's confidence is checked: if the model is confident ("I Know!"), it evaluates with an 8B model to save computation cost; if it is not confident ("I Don't Know"), it escalates the task to a larger model (LLaMA-3-70B).

Towards Fast Multilingual LLM Inference: Speculative Decoding and Specialized Drafters — [13] p.1

nd Specialized Drafters

eung[2] **Du-Seong Chang**[2] **Se-Young Yun**[1]
AI [2]KT
yunseyoung}@kaist.ac.kr
chang}@kt.com
on/Multilingual-SpecBench

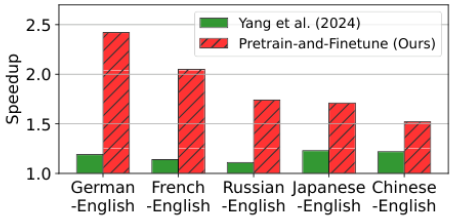


Figure 1: Speedup ratio[1] relative to the standard autoregressive greedy decoding on various multilingual datasets. Target model is Vicuna 7B v1.3 and the drafter is Vicuna 68M. Speculative greedy sampling is implemented with the drafter by Yang et al. (2024) (green) and our specialized drafter (*pretrain-and-finetune*) (red).

This bar chart compares the speedup ratio of two speculative decoding methods across five multilingual datasets: German-English, French-English, Russian-English, Japanese-English, and Chinese-English. The x-axis lists the language pairs, the y-axis represents the speedup ratio, and the bars use green to denote the method by Yang et al. (2024) and red to denote the authors' "Pretrain-and-Finetune" method. The figure conveys that the authors' specialized drafter consistently achieves a higher speedup ratio than the baseline across all tested language pairs.

Inference-Cost-Aware Dynamic Tree Construction for Efficient Inference in Large Language Models — [17] p.5

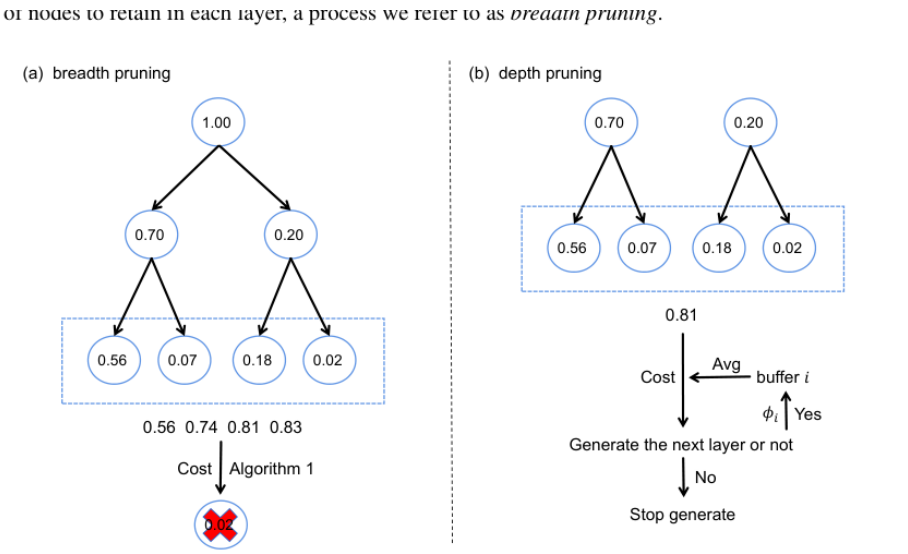
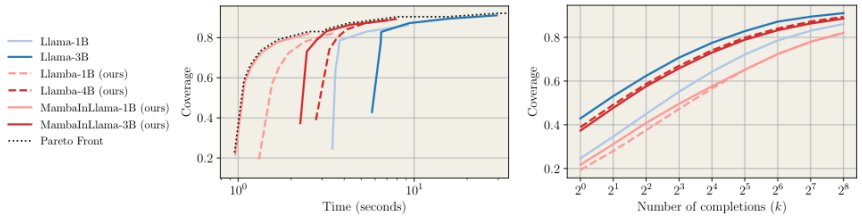


Figure 1: An illustrative example for the dynamic expansion stage, we use batch size as 1 for simplicity, general cases are tackled by averaging along batches. Each node will initially have 2 branches in the example.

The figure consists of two tree diagrams, labeled (a) breadth pruning and (b) depth pruning, illustrating different strategies for tree construction. Diagram (a) shows a hierarchical structure of nodes with numerical values connected by arrows, followed by a calculation of average values per layer and a decision process involving "Cost" and "Algorithm 1" that results in a red "X" icon. Diagram (b) shows a similar tree structure that leads to a decision flow where an average value of 0.81 is compared against a threshold ϕ_i to determine whether to generate the next layer or stop.

Thinking Slow, Fast: Scaling Inference Compute with Distilled Reasoners — [48] p.2



(a) Scaling with time.

(b) Scaling with number of completions.

Figure 1. **Distilled models have better coverage on MATH for most time budgets.** In **(b)**, we show the coverage as we increase the number of sampled answers k. Compared to their associated Llama baselines, our distilled models provide a lower coverage for a given k. In **(a)**, we now show the shortest time required to reach a given coverage. For each curve in **(b)**, we map the k-values on the x-axis to the time required to generate that many samples for each model. Ideally, we would want to reach the highest coverage for short time budget. For a given time budget, our distilled models can generate many more completions than their respective baselines. As such, the higher throughput of our models, shown in Figure 2, allows them to overcome their lower per-sample coverage. As a result, our models push the Pareto front forward for most time budgets.

Figure 1 contains two line graphs, (a) "Scaling with time" and (b) "Scaling with number of completions," comparing the coverage of Llama baseline models against various distilled "ours" models. Graph (a) plots coverage against time in seconds on a logarithmic x-axis, showing that distilled models reach higher coverage faster and push the Pareto front forward. Graph (b) plots coverage against the number of completions (k) on a logarithmic x-axis, illustrating that while distilled models have lower coverage per individual sample, their higher throughput allows them to achieve better overall coverage as k increases.

Lookahead: An Inference Acceleration Framework for Large Language Model with Lossless Generation Accuracy — [15] p.3

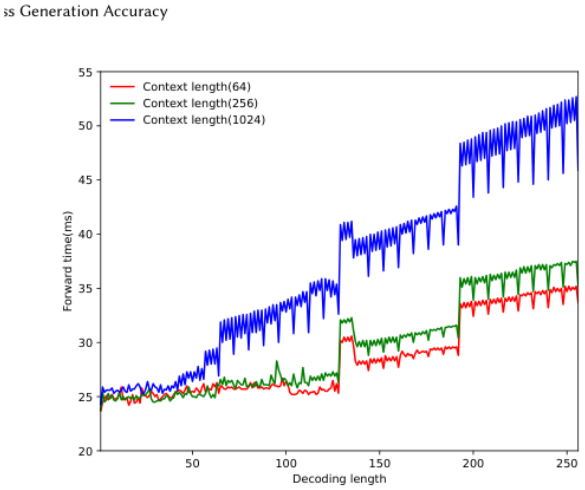


Figure 1: Decoding length's impact on the overall consuming time of LLMs' single forward process. Even the forward FLOPs is linear to the decoding length,

This line graph plots forward time in milliseconds (y-axis) against decoding length (x-axis) for

three different context lengths: 64 (red), 256 (green), and 1024 (blue). The figure conveys that as decoding length increases, the forward time grows linearly for each context length, with larger context lengths resulting in higher overall latency.

Task Scheduling for Efficient Inference of Large Language Models on Single Moderate GPU Systems — [26] p.2

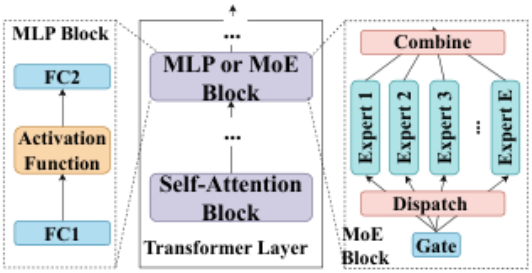


Fig. 1: The LLM architecture with dense MLP or sparse MoE blocks.

The figure depicts a hierarchical LLM architecture diagram consisting of a central "MLP or MoE Block" positioned above a "Self-Attention Block" and a "Transformer Layer." The MoE (Mixture-of-Experts) component is detailed via a sub-diagram showing a "Dispatch" mechanism that routes inputs to multiple "Expert" boxes, which are then processed by a "Combine" block.

Scaling Inference-Efficient Language Models — [45] p.1

ient Language Models

Shivaram Venkataraman [1]

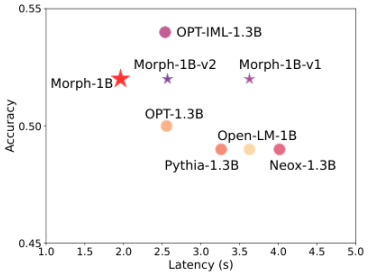
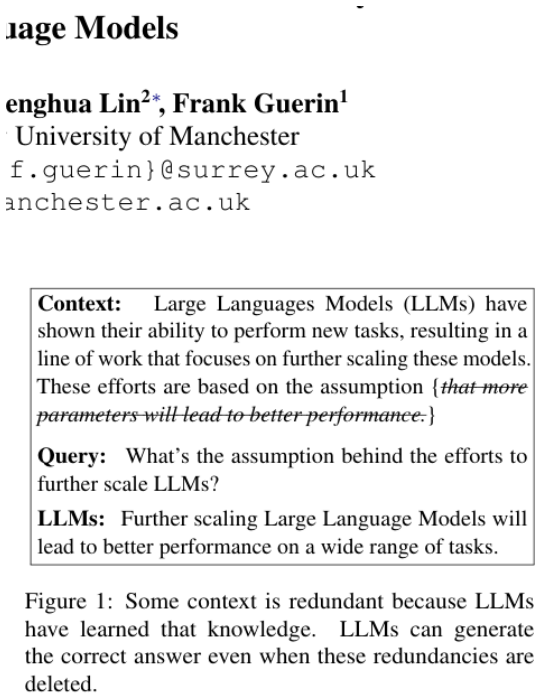


Figure 1. We train Morph-1B and its variant models on 30B tokens. The results indicate that Morph-1B maintains high accuracy on downstream tasks and achieves faster inference than open-source models and their variants. OPT-IML-1.3B achieves slightly higher performance on downstream tasks than Morph-1B since it is trained on 180B tokens (Iyer et al., 2022) and is instruction-tuned. We obtain the accuracy by evaluating models on 11 downstream tasks used by Open-LM (Gururangan et al., 2023). The inference latency is collected by using the Hugging Face `generate` function on a single NVIDIA Ampere 40GB A100 GPU with batch size 1, input length 128, and output length 256.

This scatter plot displays the relationship between model accuracy (y-axis) and inference latency in seconds (x-axis) for several 1.3B parameter language models. Data points representing Morph-1B, Morph-1B-v1, Morph-1B-v2, OPT-IML-1.3B, OPT-1.3B, Open-LM-1B, Pythia-1.3B, and Neox-1.3B are plotted to show that Morph-1B maintains high accuracy while achieving lower latency than most open-source counterparts.

Compressing Context to Enhance Inference Efficiency of Large Language Models — [5] p.1



ıage Models

enghua Lin[2]*, Frank Guerin[1]
University of Manchester
f.guerin}@surrey.ac.uk
anchester.ac.uk

Context: Large Languages Models (LLMs) have shown their ability to perform new tasks, resulting in a line of work that focuses on further scaling these models. These efforts are based on the assumption {~~*that more parameters will lead to better performance.*~~}

Query: What's the assumption behind the efforts to further scale LLMs?

LLMs: Further scaling Large Language Models will lead to better performance on a wide range of tasks.

Figure 1: Some context is redundant because LLMs have learned that knowledge. LLMs can generate the correct answer even when these redundancies are deleted.

The figure displays a text-based example consisting of three labeled sections: "Context," "Query," and "LLMs." The "Context" box contains a paragraph about scaling Large Language Models, the "Query" asks for the assumption behind scaling, and the "LLMs" section provides the corresponding answer. The figure illustrates that some context is redundant because the model can generate the correct answer even when specific information is removed.

Scaling Laws Meet Model Architecture: Toward Inference-Efficient LLMs — [46] p.1

same training budget, optimized architectures achieve up to 2.1% higher accuracy and 42% greater inference throughput compared to LLaMA-3.2.

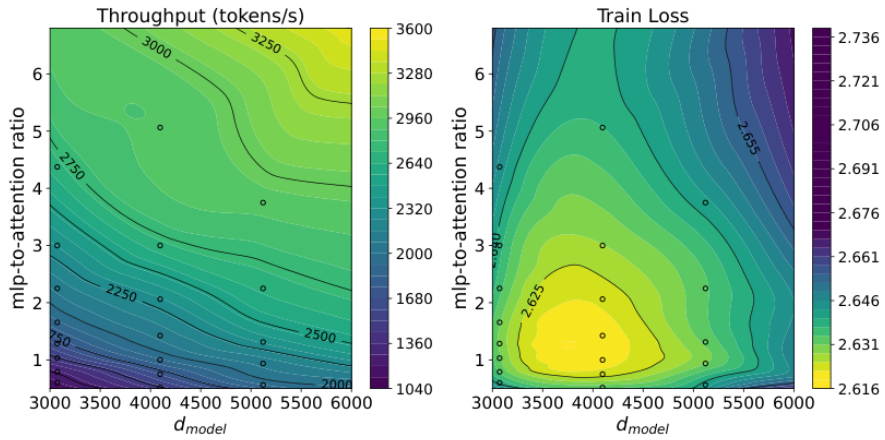


Figure 1: **Scaling sweep results.** (left) Inference throughput (tokens/s) and (right) Scaling-law-predicted training loss contours over hidden size d_{model} and mlp-to-attention ratio. Our conditional scaling law enables concurrent gains in throughput and reductions in predicted training loss under a fixed parameter budget. Dotted points indicate the architectures used to fit the scaling law.

The figure consists of two side-by-side contour plots with axes representing model size (d_{model}) and the mlp-to-attention ratio. The left plot shows inference throughput in tokens per second, illustrating how throughput increases with larger model sizes and lower mlp-to-attention ratios, while the right plot shows scaling-law-predicted training loss, illustrating how loss decreases with larger model sizes and higher mlp-to-attention ratios. Dotted points on both plots indicate the specific architectures used to fit the scaling laws.

References

[1] Wenxiao Wang, Wei Chen, Yicong Luo, Yongliu Long, Zhengkai Lin, Liye Zhang, Binbin Lin, Deng Cai, , and Xiaofei He. Model Compression and Efficient Inference for Large Language Models: A Survey. `https://arxiv.org/abs/2402.09748`, 2024.

[2] Xinyin Ma, Gongfan Fang, and Xinchao Wang. LLM-Pruner: On the Structural Pruning of Large Language Models. `https://arxiv.org/abs/2305.11627`, 2023.

[3] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A Simple and Effective Pruning Approach for Large Language Models. `https://arxiv.org/abs/2306.11695`, 2023.

[4] Zichang Liu, Jue Wang, Tri Dao, Tianyi Zhou, Binhang Yuan, Zhao Song, Anshumali Shrivastava, Ce Zhang, Yuandong Tian, Christopher Re, and Beidi Chen. Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time. `https://arxiv.org/abs/2310.17157`, 2023.

[5] Yucheng Li, Bo Dong, Chenghua Lin, and Frank Guerin. Compressing Context to Enhance Inference Efficiency of Large Language Models. `https://arxiv.org/abs/2310.06201`, 2023.

[6] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. `https://arxiv.org/abs/2310.05736`, 2023.

[7] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Í nigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. https://arxiv.org/abs/2311.18677, 2023.

[8] Georgy Tyukin. Enhancing Inference Efficiency of Large Language Models: Investigating Optimization Strategies and Architectural Innovations. https://arxiv.org/abs/2404.05741, 2024.

[9] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. https://arxiv.org/abs/2306.14048, 2023.

[10] Xuan Shen, Pu Zhao, Yifan Gong, Zhenglun Kong, Zheng Zhan, Yushu Wu, Ming Lin, Chao Wu, Xue Lin, and Yanzhi Wang. Search for Efficient Large Language Models. https://arxiv.org/abs/2409.17372, 2024.

[11] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. https://arxiv.org/abs/2211.17192, 2022.

[12] Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W. Mahoney, Amir Gholami, and Kurt Keutzer. Speculative Decoding with Big Little Decoder. https://arxiv.org/abs/2302.07863, 2023.

[13] Euiin Yi, Taehyeon Kim, Hongseok Jeung, Du-Seong Chang, and Se-Young Yun. Towards Fast Multilingual LLM Inference: Speculative Decoding and Specialized Drafters. https://arxiv.org/abs/2406.16758, 2024.

[14] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification. https://arxiv.org/abs/2305.09781, 2023.

[15] Yao Zhao, Zhitian Xie, Chen Liang, Chenyi Zhuang, and Jinjie Gu. Lookahead: An Inference Acceleration Framework for Large Language Model with Lossless Generation Accuracy. https://arxiv.org/abs/2312.12728, 2023.

[16] Sangmin Bae. Accelerating Large Language Model Inference via Early-Exiting Algorithms. https://arxiv.org/abs/2509.05915, 2025.

[17] Yinrong Hong, Zhiquan Tan, and Kai Hu. Inference-Cost-Aware Dynamic Tree Construction for Efficient Inference in Large Language Models. https://arxiv.org/abs/2510.26577, 2025.

[18] Neeraj Varshney, Agneet Chatterjee, Mihir Parmar, and Chitta Baral. Accelerating LLaMA Inference by Enabling Intermediate Layer Decoding via Instruction Tuning with LITE. https://arxiv.org/abs/2310.18581, 2023.

[19] Peizhuang Cong, Qizhi Chen, Haochen Zhao, and Tong Yang. BATON: Enhancing Batch-wise Inference Efficiency for Large Language Models via Dynamic Re-batching. https://arxiv.org/abs/2410.18701, 2024.

[20] Ozgur Guldogan, Jackson Kunde, Kangwook Lee, and Ramtin Pedarsani. Multi-Bin Batching for Increasing LLM Inference Throughput. https://arxiv.org/abs/2412.04504, 2024.

[21] Jinqi Huang, Yi Xiong, Xuebing Yu, Wenjie Huang, Entong Li, Li Zeng, and Xin Chen. SLO-Aware Scheduling for Large Language Model Inferences. https://arxiv.org/abs/2504.14966, 2025.

[22] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. S^3: Increasing GPU Utilization during Generative Inference for Higher Throughput. https://arxiv.org/abs/2306.06000, 2023.

[23] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. https://arxiv.org/abs/2312.07104, 2023.

[24] Bowen Pang, Kai Li, and Feifan Wang. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. https://arxiv.org/abs/2503.05248, 2025.

[25] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark Barrett, Joseph E. Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. https://arxiv.org/abs/2303.06865, 2023.

[26] Wenxiang Lin, Xinglin Pan, Shaohuai Shi, Xuan Wang, and Xiaowen Chu. Task Scheduling for Efficient Inference of Large Language Models on Single Moderate GPU Systems. https://arxiv.org/abs/2411.15715, 2024.

[27] Sihyeong Park, Sungryeol Jeon, Chaelyn Lee, Seokhun Jeon, Byung-Soo Kim, and Jemin Lee. A Survey on Inference Engines for Large Language Models: Perspectives on Optimization and Efficiency. https://arxiv.org/abs/2505.01658, 2025.

[28] James Pan and Guoliang Li. A Survey of LLM Inference Systems. https://arxiv.org/abs/2506.21901, 2025.

[29] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads. https://arxiv.org/abs/2401.11181, 2024.

[30] Xianzhe Dong, Tongxuan Liu, Yuting Zeng, Liangyu Liu, Yang Liu, Siyu Wu, Yu Wu, Hailong Yang, Ke Zhang, and Jing Li. HydraInfer: Hybrid Disaggregated Scheduling for Multimodal Large Language Model Serving. https://arxiv.org/abs/2505.12658, 2025.

[31] Shiyang Chen, Rain Jiang, Dezhi Yu, Jinlai Xu, Mengyuan Chao, Fanlong Meng, Chenyu Jiang, Wei Xu, , and Hang Liu. KVDirect: Distributed Disaggregated LLM Inference. https://arxiv.org/abs/2501.14743, 2025.

[32] Xing Chen, Rong Shi, Lu Zhao, Lingbin Wang, Xiao Jin, Yueqiang Chen, and Hongfeng Sun. Disaggregated Prefill and Decoding Inference System for Large Language Model Serving on Multi-Vendor GPUs. https://arxiv.org/abs/2509.17542, 2025.

[33] Akrit Mudvari, Yuang Jiang, and Leandros Tassiulas. SplitLLM: Collaborative Inference of LLMs for Model Placement and Throughput Optimization. https://arxiv.org/abs/2410.10759, 2024.

[34] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast Distributed Inference Serving for Large Language Models. https://arxiv.org/abs/2305.05920, 2023.

[35] Yanyu Chen and Ganhong Huang. GUIDE: A Global Unified Inference Engine for Deploying Large Language Models in Heterogeneous Environments. https://arxiv.org/abs/2412.04788, 2024.

[36] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhan Dong, and Yu Wang. FlashDecoding++: Faster Large Language Model Inference on GPUs. https://arxiv.org/abs/2311.01282, 2023.

[37] Ranran Zhen, Juntao Li, Yixin Ji, Zhenlin Yang, Tong Liu, Qingrong Xia, Xinyu Duan, Zhefeng Wang, Baoxing Huai, and Min Zhang. Taming the Titans: A Survey of Efficient LLM Inference Serving. https://arxiv.org/abs/2504.19720, 2025.

[38] Youngsuk Park, Kailash Budhathoki, Liangfu Chen, Jonas Kübler, Jiaji Huang, Matthäus Kleindessner, Jun Huan, Volkan Cevher, Yida Wang, and George Karypis. Inference Optimization of Foundation Models on AI Accelerators. https://arxiv.org/abs/2407.09111, 2024.

[39] Shubham Negi and Kaushik Roy. HALO: Memory-Centric Heterogeneous Accelerator with 2.5D Integration for Low-Batch LLM Inference. https://arxiv.org/abs/2510.02675, 2025.

[40] Jingyao Zhang, Jaewoo Park, Jongeun Lee, and Elaheh Sadredini. SAIL: SRAM-Accelerated LLM Inference System with Lookup-Table-based GEMV. https://arxiv.org/abs/2509.25853, 2025.

[41] Yanbiao Liang, Huihong Shi, Haikuo Shao, , and Zhongfeng Wang. AccLLM: Accelerating Long-Context LLM Inference Via Algorithm-Hardware Co-Design. https://arxiv.org/abs/2505.03745, 2025.

[42] Shuming Shi, Enbo Zhao, Deng Cai, Leyang Cui, Xinting Huang, and Huayang Li. Inferflow: an Efficient and Highly Configurable Inference Engine for Large Language Models. https://arxiv.org/abs/2401.08294, 2024.

[43] Amit Sharma. AI Accelerators for Large Language Model Inference: Architecture Analysis and Scaling Strategies. https://arxiv.org/abs/2506.00008, 2025.

[44] Jinhao Li, Jiaming Xu, Shan Huang, Yonghua Chen, Wen Li, Jun Liu, Yaoxiu Lian, Jiayi Pan, Li Ding, Hao Zhou, Yu Wang, and Guohao Dai. Large Language Model Inference Acceleration: A Comprehensive Hardware Perspective. https://arxiv.org/abs/2410.04466, 2024.

[45] Song Bian, Minghao Yan, and Shivaram Venkataraman. Scaling Inference-Efficient Language Models. https://arxiv.org/abs/2501.18107, 2025.

[46] Song Bian, Tao Yu, Shivaram Venkataraman, and Youngsuk Park. Scaling Laws Meet Model Architecture: Toward Inference-Efficient LLMs. https://arxiv.org/abs/2510.18245, 2025.

[47] Daman Arora and Andrea Zanette. Training Language Models to Reason Efficiently. https://arxiv.org/abs/2502.04463, 2025.

[48] Daniele Paliotta, Junxiong Wang, Matteo Pagliardini, Kevin Y. Li, Aviv Bick, J. Zico Kolter, Albert Gu, François Fleuret, and Tri Dao. Thinking Slow, Fast: Scaling Inference Compute with Distilled Reasoners. https://arxiv.org/abs/2502.20339, 2025.

[49] Yiran Zhao, Shengyang Zhou, Zijian Wu, Tongyan Hu, Yuhui Xu, Rengan Dou, Kenji Kawaguchi, Shafiq Joty, Junnan Li, and Michael Qizhe Shieh. Gradually Compacting Large Language Models for Reasoning Like a Boiling Frog. https://arxiv.org/abs/2602.04919, 2026.

[50] Bo-Wei Chen, Chung-Chi Chen, and An-Zi Yen. Confidence-Driven Multi-Scale Model Selection for Cost-Efficient Inference. https://arxiv.org/abs/2602.22090, 2026.

[51] Zixuan Zhou, Xuefei Ning, Ke Hong, Tianyu Fu, Jiaming Xu, Shiyao Li, Yuming Lou, Luning Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiao-Ping Zhang, Yuhan Dong, and Yu Wang. A Survey on Efficient Inference for Large Language Models. https://arxiv.org/abs/2404.14294, 2024.

[52] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. https://arxiv.org/abs/2305.05176, 2023.

[53] Zhixiong Chen, Bingjie Zhu, Jiangzhou Wang, Hyundong Shin, Arumugam Nallanathan, , and Dusit Niyato. Network Edge Inference for Large Language Models: Principles, Techniques, and Opportunities. https://arxiv.org/abs/2604.22906, 2026.